Universidad Anáhuac Mayab

Desarrollo e Integración de una Aplicación con Base de Datos NoSQL en la Nube

Red de estudiantes UniRed sobre Neo4j AuraDB (modelo de grafo)

Jennifer Briceño

Bases de Datos Avanzadas

Profesor: Fernando Martínez Chacón

31 de octubre de 2026

1. Introducción y dominio del problema

UniRed es una red social académica para estudiantes universitarios. Su propósito es ayudar a cada estudiante a encontrar compañeros de equipo y a ampliar su círculo de contactos a partir de lo que ya tiene en común con otros: amigos, materias que cursa y habilidades que domina.

El caso de uso responde a preguntas que son naturales en una universidad y que dependen de las conexiones entre las personas más que de los datos aislados de cada una:

• ¿A quién podría conocer? Los amigos de mis amigos que todavía no son mis amigos.
• ¿Cómo estoy conectado con otra persona? La cadena más corta de amistades entre dos estudiantes.
• ¿Con quién formar equipo en una materia? Compañeros que la cursan y que aportan habilidades que yo no tengo.
• ¿Quiénes son los estudiantes más conectados y qué habilidades abundan en la comunidad?

La aplicación es una herramienta de consola escrita en Python que usa el driver oficial de Neo4j y se conecta a una instancia de Neo4j AuraDB, un servicio administrado en la nube. Incluye las operaciones CRUD completas (crear, leer, actualizar y eliminar) sobre estudiantes y sobre las relaciones entre ellos, además de ocho consultas propias de un grafo. Para las pruebas se carga un conjunto de datos ficticios: 14 estudiantes, 8 materias, 16 habilidades, 19 amistades, 35 inscripciones y 44 relaciones de dominio de habilidades.

2. Justificación del paradigma

2.1 La naturaleza del problema

En UniRed el valor está en las relaciones. Las preguntas del apartado anterior se contestan recorriendo conexiones: de un estudiante a sus amigos, de ahí a los amigos de ellos, o de una materia a quienes la cursan y de ahí a sus habilidades. Además, la profundidad del recorrido no se conoce de antemano: «¿cuántos saltos separan a dos personas?» puede ser 1, 3 o 6. Este tipo de datos describe una red, y el modelo de grafo de propiedades, con nodos y relaciones como elementos de primera clase, es la representación más directa.

2.2 Comparación con un esquema relacional

Criterio	Relacional (SQL)	Grafo (Neo4j)
Representación de las conexiones	Llaves foráneas y una tabla intermedia (por ejemplo, amistad). Cada consulta reconstruye la conexión con un JOIN.	La relación se guarda como un elemento propio, enlazado directamente a sus dos nodos.
Consultas de varios saltos	Un JOIN por cada salto. Con profundidad variable se necesita una consulta recursiva.	Un patrón declarativo, por ejemplo -[:AMIGO_DE*1..3]-, sin importar la profundidad.
Camino más corto	WITH RECURSIVE escrito a mano, con control manual de ciclos y de profundidad.	Función integrada shortestPath().
Costo de recorrer	Crece con el tamaño de las tablas unidas y depende de los índices.	Depende de la porción del grafo que se visita, no del total de datos (adyacencia sin índices).
Cambios en el modelo	ALTER TABLE y migraciones para agregar tipos de relación o columnas.	Se agregan etiquetas, relaciones y propiedades nuevas sin migrar lo existente.
Legibilidad	La intención se pierde entre JOIN y condiciones.	El patrón dibujado en la consulta se parece al dibujo del dominio.

2.3 Comparación de código

Amigos de amigos que aún no son amigos, en SQL (suponiendo que cada amistad se guarda en las dos direcciones):

```
SELECT a2.id_amigo, COUNT(*) AS amigos_en_comun
FROM amistad a1
JOIN amistad a2 ON a2.id_estudiante = a1.id_amigo
WHERE a1.id_estudiante = :yo
  AND a2.id_amigo <> :yo
  AND NOT EXISTS (SELECT 1 FROM amistad x
                  WHERE x.id_estudiante = :yo AND x.id_amigo = a2.id_amigo)
GROUP BY a2.id_amigo;
```

Camino más corto entre dos estudiantes, en SQL (PostgreSQL):

```
WITH RECURSIVE ruta(id, camino, saltos) AS (
  SELECT id_amigo, ARRAY[id_estudiante, id_amigo], 1
  FROM amistad WHERE id_estudiante = :origen
  UNION ALL
  SELECT a.id_amigo, r.camino || a.id_amigo, r.saltos + 1
  FROM ruta r JOIN amistad a ON a.id_estudiante = r.id
  WHERE a.id_amigo <> ALL (r.camino) AND r.saltos < 6
)
SELECT camino FROM ruta WHERE id = :destino ORDER BY saltos LIMIT 1;
```

Las mismas dos preguntas en Cypher, tal como están en la aplicación:

```
MATCH (yo:Estudiante {correo: $correo})-[:AMIGO_DE]-(:Estudiante)
      -[:AMIGO_DE]-(sugerido:Estudiante)
WHERE sugerido <> yo AND NOT (yo)-[:AMIGO_DE]-(sugerido)
RETURN sugerido.correo AS correo, sugerido.nombre AS nombre, count(*) AS amigos_en_comun
ORDER BY amigos_en_comun DESC, nombre
LIMIT $limite

MATCH (a:Estudiante {correo: $origen}), (b:Estudiante {correo: $destino})
MATCH ruta = shortestPath((a)-[:AMIGO_DE*..6]-(b))
RETURN [n IN nodes(ruta) | n.nombre] AS camino, length(ruta) AS grados
```

La versión en Cypher es más corta, no requiere duplicar cada amistad en dos filas y deja el límite de profundidad en una sola línea. En la versión recursiva de SQL hay que cuidar los ciclos y el costo crece con la cantidad de rutas exploradas.

2.4 Por qué grafo y no clave valor

Un almacén de clave valor como Redis es excelente para búsquedas directas por llave, cachés y sesiones, pero no puede recorrer conexiones. Para obtener los amigos de los amigos de un estudiante, la aplicación tendría que pedir la lista de amigos, luego pedir la lista de cada uno de ellos y combinar los resultados por su cuenta, con muchas idas y vueltas a la red. En un grafo, esa lógica se ejecuta dentro del motor, en una sola consulta.

2.5 Límites del enfoque

El modelo de grafo no es la mejor opción para todo. Si la aplicación se centrara en reportes agregados sobre millones de filas tabulares, o en transacciones con un esquema muy rígido y reglas de integridad declarativas complejas, un sistema relacional sería más adecuado. Para UniRed, donde el núcleo son las conexiones y los recorridos, el grafo es la elección correcta.

2.6 Consistencia y garantías

Neo4j es un motor transaccional con propiedades ACID: cada escritura de la aplicación se ejecuta como una transacción que se confirma completa o se revierte. La instancia gratuita de AuraDB corre como un único servidor, por lo que las lecturas siempre ven las últimas escrituras. Con varias instancias en clúster, el driver mantiene la consistencia causal mediante marcadores de transacción (bookmarks), que su método execute_query administra automáticamente.

3. Arquitectura de la solución

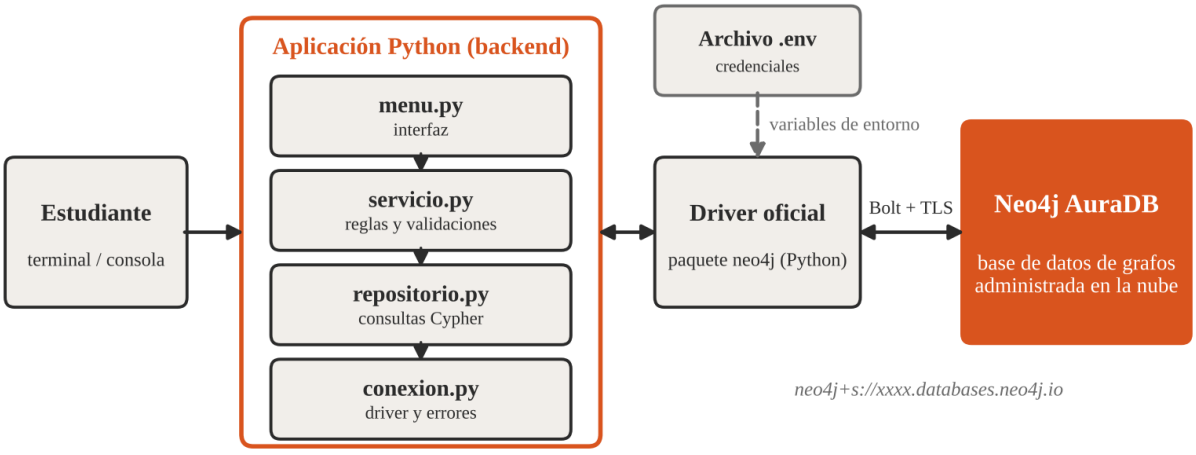


Figura 1. Diagrama de bloques de la arquitectura.

La aplicación se divide en capas, y cada una solo conoce a la siguiente:

Componente	Archivo	Responsabilidad
Interfaz	app/menu.py, app/entrada.py	Menús de consola, lectura segura del teclado y presentación de resultados.
Servicio	app/servicio.py, app/validaciones.py	Reglas de negocio: valida datos, comprueba que los estudiantes y materias existan y evita duplicados.
Repositorio	app/repositorio.py, app/consultas.py	Catálogo de consultas Cypher y métodos que las ejecutan con parámetros.
Conexión	app/conexion.py, app/config.py	Crea el driver, verifica la conexión, ejecuta las consultas y traduce las excepciones del driver.
Datos iniciales	scripts/seed.py, scripts/datos_iniciales.py	Crea las restricciones y carga los datos de ejemplo de forma repetible.

3.1 Flujo de una solicitud

Cuando la persona elige «Sugerir amigos», el menú pide el correo y llama al servicio. El servicio valida el formato del correo y comprueba en la base que el estudiante exista. Luego el repositorio ejecuta la consulta Cypher con el correo como parámetro, a través de la conexión. El driver envía la consulta por el protocolo Bolt cifrado con TLS hacia AuraDB, la base la resuelve recorriendo el grafo y devuelve las filas, que el menú muestra como tabla.

3.2 Seguridad y despliegue en la nube

La base de datos está en Neo4j AuraDB, un servicio administrado (DaaS). La aplicación rechaza al arrancar cualquier dirección local (localhost) y solo acepta URI cifradas del tipo neo4j+s://. Las credenciales nunca están en el código: se leen de las variables de entorno NEO4J_URI, NEO4J_USERNAME, NEO4J_PASSWORD y NEO4J_DATABASE, que se guardan en un archivo .env excluido del repositorio con .gitignore. Todas las consultas usan parámetros ($correo, $limite, etc.) y nunca se concatenan datos del usuario al texto de la consulta; la única excepción es la profundidad de la vecindad, que Cypher no permite parametrizar y que se valida como entero de 1 a 4 antes de usarse.

3.3 Manejo de errores

Los errores esperados (dato inválido, estudiante inexistente, correo duplicado, credenciales incorrectas, instancia pausada o sin internet) se convierten en excepciones propias con un mensaje claro. El menú los muestra y vuelve a la pantalla anterior sin cerrarse. Si se viola una restricción de unicidad, el driver lo reporta y la aplicación lo traduce a un aviso de duplicado.

3.4 Estructura del repositorio

```
red_estudiantil_neo4j/
  main.py                    punto de entrada
  requirements.txt           neo4j, python-dotenv
  .env.example               plantilla de variables de entorno
  app/
    config.py                lectura y validación de variables de entorno
    conexion.py              driver oficial de Neo4j
    consultas.py             catálogo de consultas Cypher
    repositorio.py           ejecuta las consultas
    servicio.py              reglas de negocio
    validaciones.py          validación y normalización de datos
    menu.py, entrada.py      interfaz de consola
    errores.py               excepciones propias
  scripts/
    seed.py                  carga de datos iniciales
    datos_iniciales.py       datos ficticios de ejemplo
    verificar_conexion.py    prueba de conexión
  pruebas/test_aplicacion.py pruebas unitarias
```

4. Diseño del modelo NoSQL

El modelo es un grafo de propiedades con tres etiquetas de nodo y tres tipos de relación.

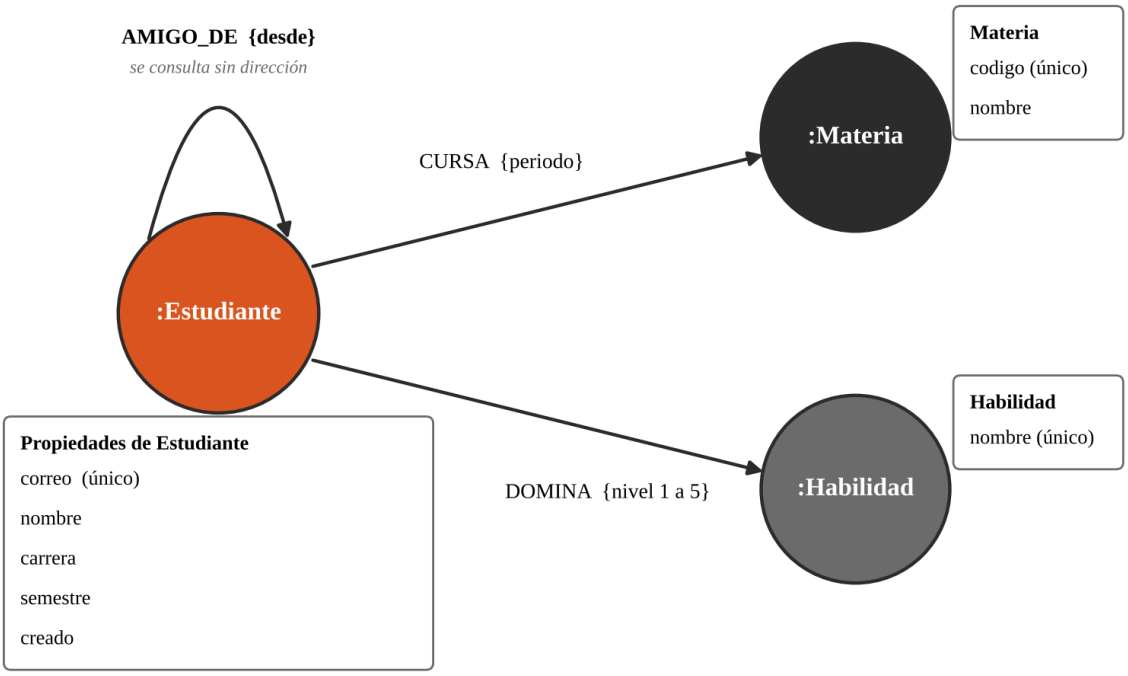


Figura 2. Diagrama del grafo del dominio: etiquetas, propiedades y relaciones.

4.1 Etiquetas de nodo y propiedades

Etiqueta	Propiedades	Descripción
Estudiante	correo (único, minúsculas), nombre, carrera, semestre (1 a 12), creado (fecha)	Persona de la red. El correo es su identificador natural y no se modifica.
Materia	codigo (único, ej. BDA-401), nombre	Materia que se cursa en el periodo.
Habilidad	nombre (único, minúsculas)	Habilidad compartida entre estudiantes (python, sql, figma). Es un nodo para poder contarla y recorrerla.

4.2 Relaciones

Relación	Dirección	Propiedades	Significado
AMIGO_DE	(Estudiante) → (Estudiante). Se guarda en un solo sentido y se consulta sin dirección.	desde (fecha)	Amistad. Es simétrica, así que se evita duplicarla en los dos sentidos.
CURSA	(Estudiante) → (Materia)	periodo (ej. 2026-Otoño)	El estudiante cursa la materia en ese periodo.
DOMINA	(Estudiante) → (Habilidad)	nivel (1 a 5)	Qué tan bien domina la habilidad.

4.3 Restricciones e índices

Se crean tres restricciones de unicidad, que además generan un índice automático sobre la propiedad. Esto hace rápidas las búsquedas de un nodo por su llave (el punto de partida de casi todas las consultas) y garantiza que no existan dos estudiantes con el mismo correo.

```
CREATE CONSTRAINT estudiante_correo_unico
  IF NOT EXISTS
  FOR (e:Estudiante) REQUIRE e.correo IS UNIQUE
CREATE CONSTRAINT materia_codigo_unico
  IF NOT EXISTS
  FOR (m:Materia) REQUIRE m.codigo IS UNIQUE
CREATE CONSTRAINT habilidad_nombre_unico
  IF NOT EXISTS
  FOR (h:Habilidad) REQUIRE h.nombre IS UNIQUE
```

Neo4j Community y AuraDB Free no ofrecen restricciones de existencia de propiedades, por lo que la obligatoriedad de nombre, carrera y semestre se valida en la capa de servicio antes de escribir.

4.4 Decisiones de diseño

Habilidad como nodo y no como lista. Guardar las habilidades como una lista dentro de Estudiante impediría preguntar de forma natural «quién más domina python» o «qué habilidades faltan a este equipo». Como nodo compartido, la habilidad conecta a todos quienes la dominan y permite agregarlas con una sola consulta.

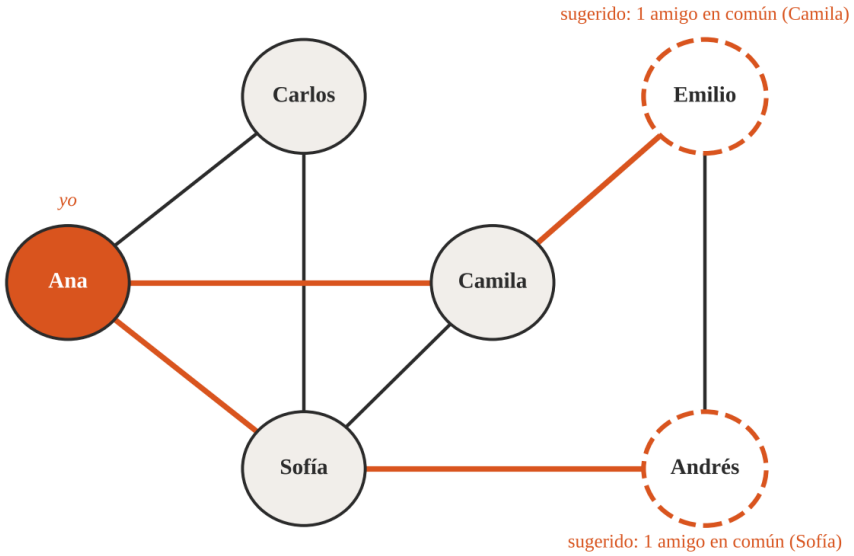
Propiedades en las relaciones. El periodo de CURSA y el nivel de DOMINA describen la relación, no a uno de los dos nodos, por lo que viven en la relación. Así un mismo estudiante puede cursar materias en distintos periodos y dominar cada habilidad en un nivel distinto.

Amistad en un solo sentido, consulta sin dirección. MERGE con un patrón sin dirección no crea una segunda relación si ya existe una en cualquier sentido, y las consultas la recorren sin

importar el sentido. Se evitan duplicados y se mantiene el recuento correcto de amigos.

Convención de nombres. Etiquetas en singular y con mayúscula inicial (Estudiante), tipos de relación en mayúsculas con guion bajo (AMIGO_DE) y propiedades en minúsculas.

4.5 Ejemplo con datos de la carga inicial



Líneas continuas: AMIGO_DE. En naranja: recorrido de 2 saltos desde Ana (círculos punteados: sugerencias).

Figura 3. Fragmento del grafo cargado. Recorrido de dos saltos de la consulta de sugerencia de amigos desde Ana.

4.6 Rendimiento y persistencia

Las consultas empiezan siempre en un nodo localizado por una propiedad con índice y recorren desde ahí, de modo que el costo depende de los vecinos visitados y no del total de nodos. Los recorridos de longitud variable están acotados (máximo 6 en el camino más corto y 4 en la vecindad) y las listas de resultados usan LIMIT. La persistencia la administra el servicio AuraDB: los datos se conservan entre ejecuciones de la aplicación y no dependen del equipo de quien la usa. Las instancias gratuitas se pausan tras un periodo sin actividad y deben reanudarse desde la consola de Aura antes de usarse.

5. Catálogo de consultas

Todas las consultas están en el archivo app/consultas.py y se ejecutan con el método execute_query del driver oficial, que abre la transacción, la confirma o revierte y administra los marcadores de consistencia. La lectura usa RoutingControl.READ y la escritura RoutingControl.WRITE.

```
resultado = driver.execute_query(
    consulta,
    parameters_={...},
    routing_=RoutingControl.READ,   # o WRITE
    database_=base_de_datos,
)
filas = [registro.data() for registro in resultado.records]
```

5.1 CRUD de estudiantes

Crear

Crea el nodo. Si el correo ya existe, la restricción de unicidad lo rechaza.

```
CREATE (e:Estudiante {
    correo: $correo, nombre: $nombre, carrera: $carrera,
    semestre: $semestre, creado: toString(date())
})
RETURN e {.*} AS estudiante
```

Método del repositorio: `crear_estudiante(correo, nombre, carrera, semestre)`

Leer uno, listar y buscar

Lectura por llave, listado completo y búsqueda parcial por nombre.

```
MATCH (e:Estudiante {correo: $correo})
RETURN e {.*} AS estudiante


MATCH (e:Estudiante)
RETURN e {.*} AS estudiante
ORDER BY e.nombre


MATCH (e:Estudiante)
WHERE toLower(e.nombre) CONTAINS toLower($texto)
RETURN e {.*} AS estudiante
ORDER BY e.nombre
```

Método del repositorio: `obtener_estudiante, listar_estudiantes, buscar_estudiantes`

Actualizar

SET con += mezcla solo las propiedades enviadas y deja intactas las demás.

```
MATCH (e:Estudiante {correo: $correo})
SET e += $cambios
RETURN e {.*} AS estudiante
```

Método del repositorio: `actualizar_estudiante(correo, cambios)`

Eliminar

DETACH DELETE borra el nodo y todas sus relaciones; devuelve cuántas tenía.

```
MATCH (e:Estudiante {correo: $correo})
WITH e, e.nombre AS nombre, COUNT { (e)--() } AS relaciones
DETACH DELETE e
RETURN nombre, relaciones
```

Método del repositorio: `eliminar_estudiante(correo)`

5.2 Materias y relaciones

Materias

Alta, lectura y listado de materias.

```
CREATE (m:Materia {codigo: $codigo, nombre: $nombre})
RETURN m {.*} AS materia
```

```
MATCH (m:Materia {codigo: $codigo})
RETURN m {.*} AS materia
```

```
MATCH (m:Materia)
RETURN m {.*} AS materia
ORDER BY m.codigo
```

Método del repositorio: `crear_materia, obtener_materia, listar_materias`

Amistades

MERGE sin dirección evita duplicar la amistad; también se consulta y se elimina.

```
MATCH (a:Estudiante {correo: $correo_a}), (b:Estudiante {correo: $correo_b})
MERGE (a)-[r:AMIGO_DE]-(b)
ON CREATE SET r.desde = toString(date())
RETURN a.nombre AS a, b.nombre AS b


MATCH (a:Estudiante {correo: $correo_a})-[r:AMIGO_DE]-(b:Estudiante {correo: $correo_b})
DELETE r
RETURN a.nombre AS a, b.nombre AS b


MATCH (:Estudiante {correo: $correo})-[r:AMIGO_DE]-(amigo:Estudiante)
RETURN amigo {.correo, .nombre, .carrera, .semestre} AS estudiante, r.desde AS desde
ORDER BY amigo.nombre


MATCH (:Estudiante {correo: $correo_a})-[r:AMIGO_DE]-
      (:Estudiante {correo: $correo_b})
RETURN count(r) AS total
```

Método del repositorio: `agregar_amistad, eliminar_amistad, listar_amigos, existe_amistad`

Inscripción en materias

Relación CURSA con la propiedad periodo; se puede dar de baja y consultar.

```
MATCH (e:Estudiante {correo: $correo}), (m:Materia {codigo: $codigo})
MERGE (e)-[r:CURSA]->(m)
ON CREATE SET r.periodo = $periodo
RETURN e.nombre AS estudiante, m.nombre AS materia, r.periodo AS periodo


MATCH (:Estudiante {correo: $correo})-[r:CURSA]->(m:Materia {codigo: $codigo})
DELETE r
RETURN m.nombre AS materia


MATCH (:Estudiante {correo: $correo})-[r:CURSA]->(m:Materia)
RETURN m {.*} AS materia, r.periodo AS periodo
ORDER BY m.codigo
```

Método del repositorio: `inscribir_materia, dar_de_baja_materia, materias_de`

Habilidades

MERGE crea la habilidad la primera vez y la reutiliza; SET actualiza el nivel si ya existía la relación.

```
MATCH (e:Estudiante {correo: $correo})
MERGE (h:Habilidad {nombre: $habilidad})
MERGE (e)-[r:DOMINA]->(h)
SET r.nivel = $nivel
RETURN e.nombre AS estudiante, h.nombre AS habilidad, r.nivel AS nivel


MATCH (:Estudiante {correo: $correo})-[r:DOMINA]->(h:Habilidad {nombre: $habilidad})
DELETE r
RETURN h.nombre AS habilidad


MATCH (:Estudiante {correo: $correo})-[r:DOMINA]->(h:Habilidad)
RETURN h.nombre AS habilidad, r.nivel AS nivel
ORDER BY r.nivel DESC, h.nombre
```

Método del repositorio: `agregar_habilidad, quitar_habilidad, habilidades_de`

5.3 Consultas especializadas del grafo

Sugerir amigos (recorrido de 2 saltos)

Amigos de mis amigos que todavía no son mis amigos, ordenados por cantidad de amigos en común.

```
MATCH (yo:Estudiante {correo: $correo})-[:AMIGO_DE]-(:Estudiante)
      -[:AMIGO_DE]-(sugerido:Estudiante)
WHERE sugerido <> yo AND NOT (yo)-[:AMIGO_DE]-(sugerido)
RETURN sugerido.correo AS correo, sugerido.nombre AS nombre, count(*) AS amigos_en_comun
ORDER BY amigos_en_comun DESC, nombre
LIMIT $limite
```

Método del repositorio: `sugerir_amigos(correo, limite)`

Camino más corto

Cadena mínima de amistades entre dos estudiantes, hasta 6 grados. Devuelve los nombres en orden.

```
MATCH (a:Estudiante {correo: $origen}), (b:Estudiante {correo: $destino})
MATCH ruta = shortestPath((a)-[:AMIGO_DE*..6]-(b))
RETURN [n IN nodes(ruta) | n.nombre] AS camino, length(ruta) AS grados
```

Método del repositorio: `camino_mas_corto(origen, destino)`

Amigos en común

Estudiantes que son amigos de ambos.

```
MATCH (:Estudiante {correo: $correo_a})-[:AMIGO_DE]-(comun:Estudiante)
      -[:AMIGO_DE]-(:Estudiante {correo: $correo_b})
RETURN DISTINCT comun.correo AS correo, comun.nombre AS nombre
ORDER BY nombre
```

Método del repositorio: `amigos_en_comun(correo_a, correo_b)`

Vecindad

Todos los alcanzables hasta N grados, con su distancia mínima. __GRADOS__ se sustituye por un entero de 1 a 4 ya validado.

```
MATCH ruta = (yo:Estudiante {correo: $correo})
             -[:AMIGO_DE*1..__GRADOS__]-(otro:Estudiante)
WHERE otro <> yo
RETURN otro.correo AS correo, otro.nombre AS nombre, min(length(ruta)) AS grados
ORDER BY grados, nombre
```

Método del repositorio: `vecindad(correo, grados)`

Compañeros de materia

Quienes cursan las mismas materias que yo, con la lista de materias compartidas.

```
MATCH (yo:Estudiante {correo: $correo})-[:CURSA]->(m:Materia)<-[:CURSA]-(c:Estudiante)
WHERE c <> yo
RETURN c.correo AS correo, c.nombre AS nombre,
       collect(m.nombre) AS materias_en_comun, count(m) AS total
ORDER BY total DESC, nombre
```

Método del repositorio: `companeros_de_materia(correo)`

Sugerir equipo para una materia

Compañeros de una materia que dominan habilidades que yo no tengo, ordenados por la suma de sus niveles.

```
MATCH (yo:Estudiante {correo: $correo})-[:CURSA]->(:Materia {codigo: $codigo})
      <-[:CURSA]-(c:Estudiante)
WHERE c <> yo
MATCH (c)-[d:DOMINA]->(h:Habilidad)
WHERE NOT (yo)-[:DOMINA]->(h)
RETURN c.correo AS correo, c.nombre AS nombre,
       collect(h.nombre) AS habilidades_que_aporta, sum(d.nivel) AS puntaje
ORDER BY puntaje DESC, nombre
LIMIT $limite
```

Método del repositorio: `sugerir_equipo(correo, codigo, limite)`

Estudiantes más conectados

Grado de cada nodo: cuenta de amigos distintos.

```
MATCH (e:Estudiante)
OPTIONAL MATCH (e)-[:AMIGO_DE]-(amigo:Estudiante)
RETURN e.nombre AS nombre, e.carrera AS carrera, count(DISTINCT amigo) AS amigos
ORDER BY amigos DESC, nombre
LIMIT $limite
```

Método del repositorio: `mas_conectados(limite)`

Habilidades más populares

Agrupa por habilidad y calcula cuántos estudiantes la dominan y su nivel promedio.

```
MATCH (:Estudiante)-[d:DOMINA]->(h:Habilidad)
RETURN h.nombre AS habilidad, count(*) AS estudiantes, round(avg(d.nivel), 1) AS nivel_promedio
ORDER BY estudiantes DESC, habilidad
LIMIT $limite
```

Método del repositorio: `habilidades_populares(limite)`

5.4 Carga inicial

El script scripts/seed.py crea las restricciones y ejecuta cinco consultas con UNWIND, que insertan listas completas en una sola transacción. Usan MERGE, por lo que se pueden ejecutar varias veces sin duplicar datos.

```
UNWIND $estudiantes AS s
MERGE (e:Estudiante {correo: s.correo})
ON CREATE SET e.creado = toString(date())
SET e.nombre = s.nombre, e.carrera = s.carrera, e.semestre = s.semestre


UNWIND $amistades AS p
MATCH (a:Estudiante {correo: p.a}), (b:Estudiante {correo: p.b})
MERGE (a)-[r:AMIGO_DE]-(b)
ON CREATE SET r.desde = p.desde


UNWIND $inscripciones AS i
MATCH (e:Estudiante {correo: i.correo}), (m:Materia {codigo: i.codigo})
MERGE (e)-[r:CURSA]->(m)
SET r.periodo = i.periodo


UNWIND $habilidades AS h
MATCH (e:Estudiante {correo: h.correo})
MERGE (x:Habilidad {nombre: h.habilidad})
MERGE (e)-[r:DOMINA]->(x)
SET r.nivel = h.nivel
```

Referencias

Neo4j. (s. f.). *Cypher manual*. https://neo4j.com/docs/cypher-manual/current/

Neo4j. (s. f.). *Neo4j Aura documentation*. https://neo4j.com/docs/aura/

Neo4j. (s. f.). *Neo4j Python driver manual*. https://neo4j.com/docs/python-manual/current/

Robinson, I., Webber, J., y Eifrem, E. (2015). *Graph databases* (2.ª ed.). O'Reilly Media.